Recruitment Assist Technical Documentation

Full-stack AI recruitment application

System Purpose

Recruitment Assist helps recruiters manage job descriptions, candidate profiles, resume/JD matching, assessments, interviews, vendor fulfilment, dashboards, reports, and AI-assisted recruiter workflows.

Core Runtime

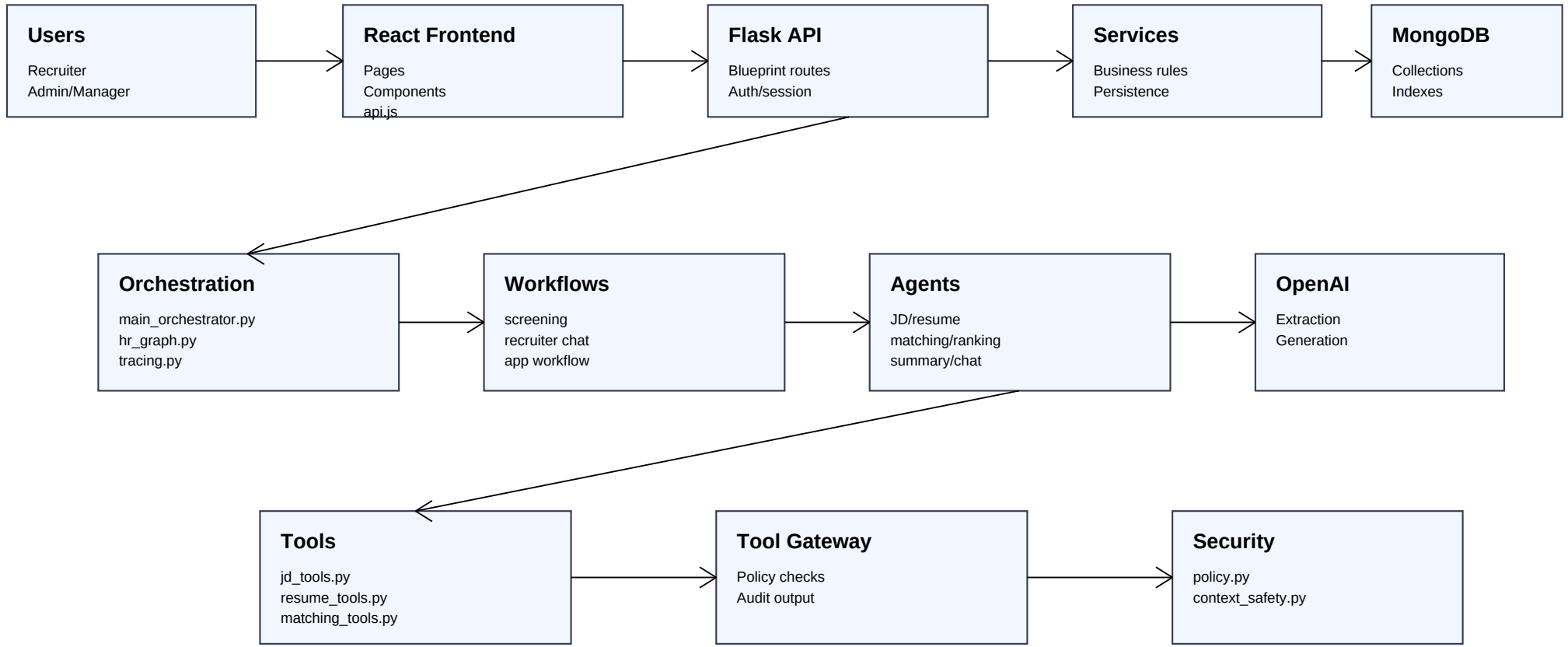
React	Flask	MongoDB	OpenAI	Agents	SMTP/Twilio
User interface	API backend	Persistence	AI extraction	Workflow logic	Messaging

Request Flow

- Recruiter/Admin uses the React frontend.
- frontend/src/api.js sends requests to Flask API endpoints.
- Routes call services, workflows, agents, or controlled tools.
- Services read/write MongoDB collections and integrate communication utilities.
- Agents use OpenAI and local context to parse, match, rank, summarize, and assist.

Architecture Diagram

High-level application structure



The application keeps UI, API, business services, persistence, and agent behavior separated.

That separation makes the system easier to document, test, and extend.

Introduction

Technical documentation

- Recruitment Assist is a full-stack AI recruitment platform for managing clients, job descriptions, candidates, resume screening, assessments, interviews, vendors, dashboards, reports, and recruiter assistance.

- The application combines a React frontend, Flask backend, MongoDB persistence layer, OpenAI-based extraction/generation, and local agent workflow modules.

Technology Stack

Technical documentation

- Frontend: React 18, React Router DOM, Create React App, custom CSS, Flatpickr, jsPDF, xlsx, docx, and file-saver.
- Backend: Python, Flask, Flask-CORS, Werkzeug, python-dotenv, Pydantic, and pytest.
- Database: MongoDB through PyMongo with indexes, integer counters, session tokens, audit logs, and cached dashboard metrics.
- AI/workflows: OpenAI Python SDK, LangGraph/LangChain packages, local agents, workflow modules, and a controlled tool gateway.
- Document processing: PyPDF2, PyMuPDF, pdfplumber, and python-docx.
- Communication: SMTP email and optional Twilio SMS.

Project Structure

Technical documentation

- frontend/src/pages contains the main React screens: Dashboard, Candidates, JD pages, Comparison, Interviews, Reports, Vendors, Assessments, Workflow Admin, and Profile.

- frontend/src/components contains shared layout, navigation, recruiter chat, and feedback components.

- backend/routes contains Flask blueprints grouped by feature.

- backend/services contains business logic for auth, JDs, candidates, matching, interviews, reports, vendors, assessments, fulfilment, workflow admin, and prompts.

- backend/agents contains role-specific AI agents for routing, JD parsing, resume parsing, matching, ranking, summaries, interviews, and recruiter chat.

- backend/tools contains controlled agent tools such as JD context loading, resume profile extraction, matching persistence, recruiter context, and the tool gateway.

- backend/workflows and backend/orchestration contain multi-step workflows, the main orchestrator, LangGraph state graph, and tracing.

Backend Architecture

Technical documentation

- backend/app.py loads environment variables, creates the Flask app, configures upload limits and CORS, registers feature blueprints, initializes MongoDB, and redirects non-API browser requests to the React frontend.

- Routes validate HTTP inputs and return JSON responses.

- Services implement business rules, persistence workflows, matching, fulfilment, email/SMS handling, assessment lifecycle, vendors, reporting, and workflow admin controls.

- Agents perform AI-assisted extraction, matching, ranking, summarization, interview content generation, and recruiter chat.

- database.py centralizes MongoDB access, indexes, counters, serialization, audit logs, session tokens, and dashboard metrics.

Frontend Architecture

Technical documentation

- The React frontend is a single-page application with protected routes.
- frontend/src/api.js centralizes backend communication and session token handling.
- Key screens include Dashboard, Job Description list/create/details, Candidates, Candidate Profile, Comparison, Interviews, Reports, Vendors, Assessments, Workflow Admin, Login, and Profile.
- The frontend uses page-specific CSS files under frontend/src/styles plus shared application styles.

Database Collections

Technical documentation

- users and user_session_tokens store login accounts and expiring API session tokens.
- clients and projects store account ownership metadata.
- job_descriptions store parsed JD data, categories, status, workflow state, and fulfilment counters.
- candidates store uploaded/internal/vendor candidates, resume metadata, structured profile data, category fields, and hiring status.
- comparisons store JD-candidate match records, scores, summaries, selection status, and workflow metadata.
- interviews store schedule data, email/SMS state, follow-up/cancellation content, and outcomes.
- vendors, jd_vendor_assignments, and vendor_email_logs support external vendor fulfilment and communication.
- assessments, questions, candidate_answers, and assessment_results store assessment lifecycle data.
- agent_runs tracks agentic task execution, while audit_logs stores a hash-chained audit trail.

AI and Agentic Architecture

Technical documentation

- main_orchestrator.py creates run IDs, determines task type, records status, and dispatches work.
- hr_graph.py defines graph-based HR routing and state transitions.
- screening_workflow.py runs the JD, resume, matching, ranking, and summary sequence.
- recruiter_chat_workflow.py handles recruiter messages, context retrieval, answer generation, and action proposals.
- router_agent.py, jd_agent.py, resume_agent.py, matching_agent.py, ranking_agent.py, summary_agent.py, interview_agent.py, and recruiter_agent.py split AI responsibilities by role.
- tool_gateway.py and security/policy.py provide controlled tool execution and audit-aware guardrails.

Security and Operations

Technical documentation

- Passwords use Werkzeug password hashing.
- API sessions use MongoDB-backed tokens with TTL.
- Uploaded filenames are sanitized and Flask enforces a maximum upload size.
- Audit logs use previous/current hashes to support tamper detection.
- Sensitive audit parameters are redacted before storage.
- Production should enable secure cookies, restricted CORS, HTTPS, managed secrets, controlled upload storage, and least-privilege MongoDB credentials.

API Reference Summary

Main endpoint groups

Auth/Profile	/api/login, /api/logout, /api/profile
Dashboard/Reports	/api/dashboard, /api/metrics/jd-performance, /api/reports, /api/report/email
Job Descriptions	/api/jds, /api/jds/create, /api/jds/<id>, delete endpoint
Candidates	/api/candidates, /api/candidates/<id>, /api/candidates/repair
Matching/Fulfilment	/api/compare, /api/jds/<id>/fulfilment, recalculate endpoint
Interviews	/api/interviews plus schedule, reschedule, email, follow-up, outcome, cancellation
Assessments	generate, lookup, save draft, question CRUD, preview, email, send, token, answer, submit, result
Agentic	/api/agentic/run, /api/agentic/screening, /api/agentic/runs
Vendors	/api/vendors plus JD assignment, email, history, and candidate acceptance
Workflow Admin	/api/admin/workflow-readiness, backfill, required count, categories, overrides, timeline
Audit	/api/audit/logs

Setup, Environment, Testing

Operational guidance

Setup Steps

- Create and activate a Python virtual environment.
- Install backend dependencies from backend/requirements.txt.
- Configure backend/.env with MongoDB, OpenAI, CORS, Flask, SMTP, and optional Twilio values.
- Install frontend dependencies in frontend with npm install.
- Run the Flask backend and React frontend, or use the root helper scripts when available.
- Run backend pytest tests and frontend build before handing off changes.

Important Environment Variables

- MONGODB_URI or MONGO_URI
- MONGODB_DB or MONGO_DB
- SECRET_KEY
- CORS_ORIGINS
- FRONTEND_URL
- OPENAI_API_KEY
- SMTP settings for email delivery
- Twilio settings for optional SMS
- SESSION_TOKEN_TTL_HOURS
- SEED_DEFAULT_USERS and local default user passwords

Validation Checklist

- Run backend pytest tests.
- Run frontend build.
- Manually test login, JD upload, candidate upload, matching, assessments, interviews, vendors, workflow admin, and reports.

Deployment and Troubleshooting

Production notes

Deployment Guide

- Build the React frontend with npm run build.
- Deploy Flask behind a WSGI server or managed Python platform.
- Use MongoDB Atlas or another managed MongoDB deployment.
- Keep secrets outside source control.
- Configure production CORS origins, HTTPS, secure cookies, upload storage, and retention policy.

Troubleshooting

- Database unavailable: verify URI, network access, database permissions, and Atlas IP rules.
- Login failure: verify users, password hashes, and session token storage.
- Frontend cannot call backend: verify backend port, API base URL/proxy, and CORS origins.
- Parsing issues: check file type, upload folder, parser dependencies, and extraction logs.
- AI output issues: verify OPENAI_API_KEY, prompt inputs, and JSON parser fallbacks.
- Email/SMS failures: verify SMTP/Twilio configuration and recipient fields.